CDS 302: Final Project Report

Soobin Jang Yoonjoo Lee Jimin Lee

December 9, 2024

1 Introduction

The movie industry is one of the largest and most dynamic sectors in the entertainment world. Over the years, movie production has become more complex, involving multi-million-dollar budgets, global distribution, and massive audiences. Analyzing data from the movie industry can reveal valuable insights, such as trends in movie popularity, critical reception, and production costs. In this project, we aim to explore various aspects of the movie industry using SQL and Python to understand patterns and trends within movie data.

The primary objectives of this project were as follows:

- To explore a movie database to identify key insights, such as the most popular movies, highest-rated films, and those with the largest budgets.
- To process and analyze the data using SQL queries to extract relevant information.
- To use Python for further data analysis and generate meaningful visualizations that highlight significant patterns and trends.

To achieve these objectives, we created a relational database containing various information about movies, including their titles, genres, release years, budgets, box office earnings, and ratings. We used SQL to query this data and extract meaningful results, such as the highest-grossing films, top-rated movies, and those with the largest production costs. We then visualized these insights using Python to make the findings more accessible and interpretable.

By the end of this project, we hope to gain a deeper understanding of the relationships between movie budgets, revenue, and ratings, as well as identify key factors that contribute to a movie's success. These insights will be valuable for both the movie industry and data analysts looking to explore entertainment data.

2 Database Schema and Design Choices

2.1 E-R Diagram

The E-R diagram below illustrates the relationships between the main entities in the database: *Movies*, *Directors*, *Genres*, *Actors*, *Companies*, and their intermediary tables. These relationships were used as the foundation for designing the database schema.

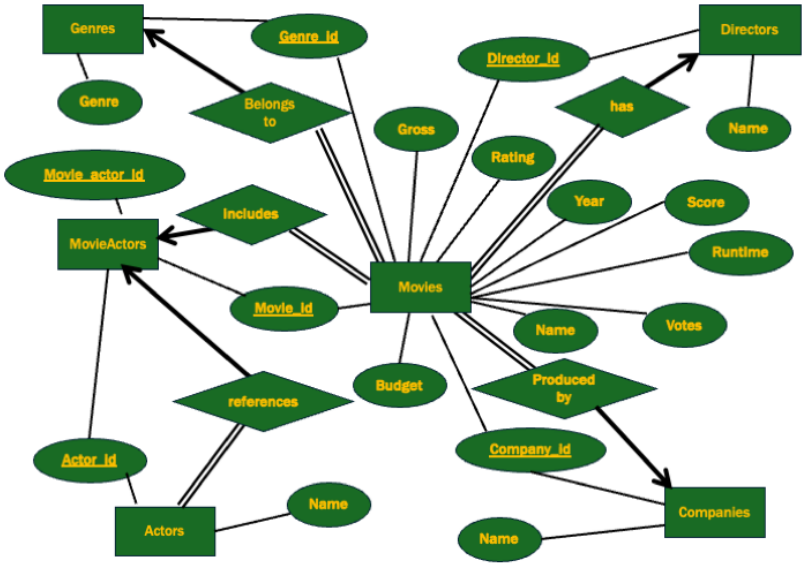


Figure 1: E-R Diagram for Movies Database

2.2 Database Schema Overview

The database schema consists of the following tables: *Movies*, *Directors*, *Genres*, *Actors*, *Companies*, and their intermediary tables. Each table represents an entity or a relationship identified during the E-R modeling process.

- **Movies:** Contains general information about each movie, such as its name, year, rating, and budget. The primary key is *MovieID*.
- **Directors:** Stores information about movie directors. The primary key is *DirectorID*.
- **Companies:** Stores details of production companies. The primary key is *CompanyID*.
- **Genres:** Stores unique movie genres. The primary key is *GenreID*.

- **Actors:** Stores information about actors. The primary key is *ActorID*.
- **Intermediary Tables:**
  - *Movie_Genres:* Links movies to genres using *MovieID* and *GenreID*.
  - *Movie_Actors:* Links movies to actors using *MovieID* and *ActorID*.

2.3 Design Choices

During the database design process, several key decisions were made:

1. **Primary Key Selection:** Unique identifiers such as *MovieID*, *DirectorID*, *CompanyID*, *GenreID*, and *ActorID* were chosen as primary keys to ensure entity integrity.
2. **Foreign Key Implementation:** Relationships between entities were represented using foreign keys. For example, *Movies* references *Directors* via *DirectorID* and *Companies* via *CompanyID*.
3. **Many-to-Many Relationships:** Intermediary tables (*Movie_Genres*, *Movie_Actors*) were introduced to handle many-to-many relationships, ensuring flexibility and scalability.
4. **Normalization:** The schema was normalized to 3NF to eliminate redundancy and maintain data integrity.

2.4 SQL DDL Script

The following SQL script defines the tables and relationships in the database:

```
CREATE TABLE Movies (
    MovieID INT PRIMARY KEY,
    Name VARCHAR(255),
    Year INT,
    Rating VARCHAR(10),
    Budget DECIMAL(15,2),
    Score DECIMAL(3,1),
    Votes INT,
    Gross DECIMAL(15,2),
    Runtime INT,
    CompanyID INT,
```

```
    DirectorID INT,
    FOREIGN KEY (CompanyID) REFERENCES Companies(CompanyID),
    FOREIGN KEY (DirectorID) REFERENCES Directors(DirectorID)
);

CREATE TABLE Directors (
    DirectorID INT PRIMARY KEY,
    Name VARCHAR(255)
);

CREATE TABLE Companies (
    CompanyID INT PRIMARY KEY,
    Name VARCHAR(255)
);

CREATE TABLE Genres (
    GenreID INT PRIMARY KEY,
    Name VARCHAR(255)
);

CREATE TABLE Movie_Genres (
    MovieID INT,
    GenreID INT,
    PRIMARY KEY (MovieID, GenreID),
    FOREIGN KEY (MovieID) REFERENCES Movies(MovieID),
    FOREIGN KEY (GenreID) REFERENCES Genres(GenreID)
);

CREATE TABLE Actors (
    ActorID INT PRIMARY KEY,
    Name VARCHAR(255)
);

CREATE TABLE Movie_Actors (
    MovieID INT,
    ActorID INT,
    PRIMARY KEY (MovieID, ActorID),
    FOREIGN KEY (MovieID) REFERENCES Movies(MovieID),
    FOREIGN KEY (ActorID) REFERENCES Actors(ActorID)
);
```

This database schema ensures data integrity, eliminates redundancy, and supports scalability while aligning with the E-R diagram.

3 ER Modeling and Schema Reduction

3.1 E-R Diagram for Movies Database

The following E-R diagram illustrates the relationships between entities such as *Movies*, *Directors*, *Genres*, *Actors*, and *Companies*. It serves as the foundation for the normalized schema.

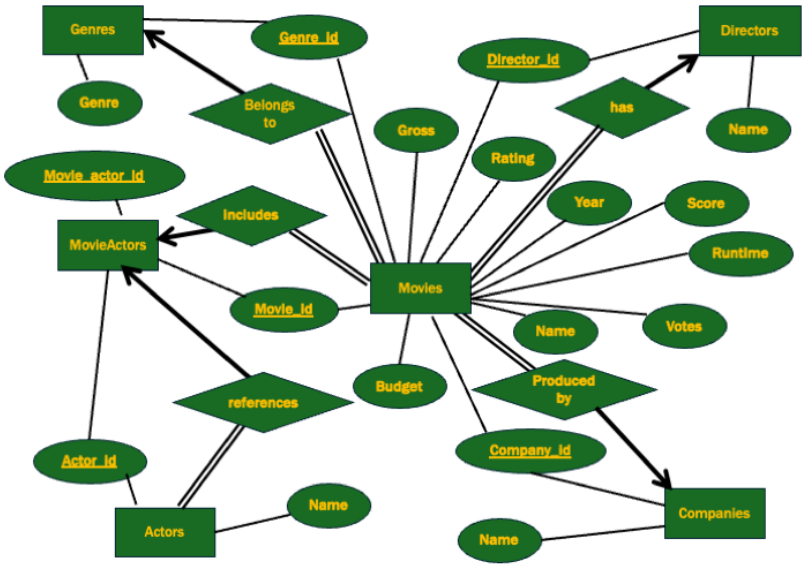


Figure 2: E-R Diagram for Movies Database

3.2 Normalized Schema Based on E-R Diagram

This document presents the normalized schema based on the provided E-R diagram. The schema is designed to ensure data integrity, minimize redundancy, and align with the relationships in the diagram.

3.3 Movies Table

Purpose: Stores basic information about each movie.

Attribute	Description
MovieID (PK)	Unique identifier for each movie
Name	Title of the movie
Year	Release year of the movie
Rating	Age rating (e.g., PG, R)
Budget	Budget of the movie
Score	IMDb score
Votes	Number of votes on IMDb
Gross	Gross revenue
Runtime	Duration in minutes
CompanyID (FK)	Foreign key referencing Companies table
DirectorID (FK)	Foreign key referencing Directors table

MovieID	Name	Year	Rating	Budget	Score	Votes	Gross	Runtime	CompanyID	DirectorID
1	Star Wars	1980	PG	18,000,000	8.7	1,200,000	538,375,067	124	1	1

3.4 Directors Table

Purpose: Stores information about directors.

Attribute	Description
DirectorID (PK)	Unique identifier for each director
Name	Director's name

Example Data:

DirectorID	Name
1	Irvin Kershner

3.5 Companies Table

Purpose: Stores production company details.

Attribute	Description
CompanyID (PK)	Unique identifier for each company
Name	Production company name

Example Data:

CompanyID	Name
1	Lucasfilm

3.6 Genres Table

Purpose: Stores unique genres.

Attribute	Description
GenreID (PK)	Unique identifier for each genre
Name	Genre name

Example Data:

GenreID	Name
1	Action
2	Sci-Fi

3.7 Movie_Genres Table

Purpose: Links movies to their genres (Many-to-Many relationship).

Attribute	Description
MovieID (FK)	Foreign key referencing Movies table
GenreID (FK)	Foreign key referencing Genres table

Example Data:

MovieID	GenreID
1	1
1	2

3.8 Actors Table

Purpose: Stores unique actor details.

Attribute	Description
ActorID (PK)	Unique identifier for each actor
Name	Actor's name

Example Data:

ActorID	Name
1	Mark Hamill
2	Carrie Fisher

3.9 Movie_Actors Table

Purpose: Links movies to their actors (Many-to-Many relationship).

Attribute	Description
MovieID (FK)	Foreign key referencing Movies table
ActorID (FK)	Foreign key referencing Actors table

Example Data:

MovieID	ActorID
1	1
1	2

This normalized schema aligns with the E-R diagram and removes redundancy while maintaining data integrity. Relationships such as *Movies-Genres*, *Movies-Actors*, and *Movies-Directors* are accurately represented with linking tables.

4 SQL Code Examples and Explanations

SQL was extensively used in our project to retrieve, manipulate, and analyze data. Below are examples of key SQL queries used, along with their purposes and explanations.

4.1 String Operations

Query:

```
SELECT
    Name AS Movie_Title, Year, Rating,
    Gross
FROM Movies
WHERE Name LIKE '%Star%' OR Name LIKE '%War%'
ORDER BY Gross DESC;
```

Explanation: This query retrieves movies with titles containing the keywords `"Star"` or `"War"`. The `LIKE` operator with the `%` wildcard enables pattern matching to identify all titles that include these keywords. The results are sorted by gross revenue in descending order. This query was instrumental in analyzing movies related to specific genres or franchises.

4.2 Aggregate Functions

Query:

```
SELECT
    Director AS Director_Name,
    COUNT(*) AS Total_Movies,
    AVG(Score) AS Avg_Score,
    SUM(Gross) AS Total_Gross
FROM Movies
GROUP BY Director
HAVING COUNT(*) > 1
ORDER BY Total_Gross DESC, Avg_Score DESC;
```

Explanation: This query calculates the total number of movies (`COUNT`), average score (`AVG`), and total gross revenue (`SUM`) for each director. The `GROUP BY` clause groups data by director, and the `HAVING` clause filters out directors with only one movie. Results are sorted by total gross revenue and then by average score. This analysis provided insights into the most impactful directors in terms of both financial and critical success.

4.3 ORDER BY & GROUP BY

Query:

```
SELECT
    d.Name AS Director_Name,
    COUNT(m.Movie_ID) AS Movie_Count,
    SUM(m.Gross) AS Total_Gross
FROM Movies m
JOIN Directors d ON m.Director_ID = d.Director_ID
GROUP BY d.Name
ORDER BY Total_Gross DESC;
```

Explanation: This query combines the `ORDER BY` and `GROUP BY` clauses to analyze directors' performance. Data is grouped by director names, and statistics such as the total number of movies and gross revenue are calculated. Results are sorted by total gross revenue in descending order, allowing us to identify the most commercially successful directors.

4.4 Most Popular Movie (Using ORDER BY)

Query:

	Movie_Title	Year	Score	Gross
1	The Shawshank Redemption	1994	9.3	28817291.0

Figure 3: The Most Popular Movie

```
SELECT
    Name AS Movie_Title,
    Year,
    Score,
    Gross
FROM Movies
WHERE Score = (SELECT MAX(Score) FROM Movies)
ORDER BY Gross DESC
LIMIT 1;
```

Explanation: This query identifies the most popular movie based on the highest score. The `WHERE` clause filters for movies with the maximum score, determined by the subquery `(SELECT MAX(Score) FROM Movies)`. In case of a tie, the `ORDER BY Gross DESC` clause ensures that the movie with the highest gross revenue is selected. The `LIMIT 1` clause restricts the result to one movie.

4.5 JOIN Clause

Query:

```
SELECT
    d.Name AS Director_Name,
    COUNT(m.Movie_ID) AS Movie_Count,
    SUM(m.Gross) AS Total_Gross
FROM Movies m
JOIN Directors d ON m.Director_ID = d.Director_ID
GROUP BY d.Name
ORDER BY Total_Gross DESC;
```

Explanation: The `JOIN` clause connects the `Movies` and `Directors` tables using the `Director_ID` field. This enables aggregation of data about movies and their directors. Grouping and ordering helped identify directors with the most significant contributions to the film industry.

4.6 HAVING Clause

Query:

```
SELECT
    m.Name AS Movie_Title,
    m.Year,
    SUM(m.Gross) AS Total_Gross
FROM Movies m
WHERE m.Year >= 2015
GROUP BY m.Movie_ID
HAVING SUM(m.Gross) > 100000000
ORDER BY Total_Gross DESC;
```

Explanation: The `HAVING` clause is used to filter grouped data. After grouping movies released after 2015, the query calculates their total gross revenue and filters for movies with gross revenue exceeding $100 million. Results are sorted by gross revenue.

4.7 WITH-Clause

Query:

```
WITH RankedMovies AS (
    SELECT
        Name, Year, Gross,
        RANK() OVER (PARTITION BY Year ORDER BY Gross
           DESC) AS Rank
    FROM Movies
)
SELECT *
FROM RankedMovies
WHERE Rank <= 3
ORDER BY Year DESC, Rank ASC;
```

Explanation: The `WITH` clause (Common Table Expression) is used to rank movies by gross revenue within each year. The `RANK()` function assigns rankings within partitions defined by `Year`. The final query retrieves the top three movies per year, ordered by year and rank.

5 Discussion of Relational Storage and ACID Compliance

5.1 Relational Storage in the Context of Our Project

Relational storage played a critical role in our project, organizing data into structured tables that represent different entities such as Movies, Directors, and Genres. These tables are connected using **Primary Keys** and **Foreign Keys**, ensuring data integrity and consistency.

For example:

- The `Movies` table includes a `Director_ID` column, which acts as a **Foreign Key** referencing the `Director_ID` in the `Directors` table. This relationship ensures that every movie is linked to a valid director, maintaining logical consistency across the database.
- To avoid duplication, related data is normalized and separated into individual tables. For instance:
  - The many-to-many relationship between Movies and Actors is managed through a `Movie_Actors` table. This intermediary table includes Foreign Keys (`Movie_ID` and `Actor_ID`) to connect the `Movies` and `Actors` tables, eliminating redundancy.

Primary Keys, such as `Movie_ID` in the `Movies` table, uniquely identify each record, while Foreign Keys establish meaningful relationships between tables. This relational approach maintains an efficient and easily manageable database structure.

5.2 ACID Compliance in the Context of Our Project

ACID compliance was a core principle in our project, ensuring that database transactions are reliable, consistent, and durable. The four principles of ACID are:

- **Atomicity:** Every transaction follows the "all or nothing" rule. For instance, when adding a director and their associated movie, both actions are performed within a single transaction. If the movie addition fails, the director is not added either, keeping the database in a consistent state.
- **Consistency:** The database always adheres to its rules and constraints. For example:

  - A movie must always reference a valid director, as the `Director_ID` in the `Movies` table is a Foreign Key pointing to a valid entry in the `Directors` table.
  - Constraints such as `NOT NULL` and Foreign Key validations enforce these relationships, preventing invalid or incomplete data from being entered.

- **Isolation:** Our design ensures that multiple transactions can run concurrently without interfering with each other. For example:
  - If multiple users are adding or updating movie data simultaneously, isolation levels prevent race conditions, ensuring consistent and accurate results.

- **Durability:** Once a transaction is successfully completed, its changes are permanently saved. For instance:
  - Even in the event of a system crash, any added movie data is safely stored and recoverable.

5.3 Ensuring Data Integrity and Consistency

Our design ensures data integrity and consistency through the following methods:

- **Relational Design:** By normalizing the database and defining relationships through Foreign Keys, we prevent data redundancy and ensure valid relationships between entities.

- **Constraints:**
  - **Primary Key constraints** ensure unique identification of each record.
  - **Foreign Key constraints** enforce valid relationships between tables.
  - **NOT NULL constraints** prevent incomplete data from being entered.

- **Transaction Management:** Operations involving multiple related updates are handled within transactions to ensure atomicity and consistency.

- **Error Handling:** If any part of a transaction fails, the entire operation is rolled back, avoiding partial updates and maintaining database integrity.

By adhering to the principles of relational storage and ACID compliance, our project successfully achieves a robust, reliable, and consistent database design that supports accuracy and stability, even under concurrent usage.

6 Additional Visualization

In this section, we present the visualizations that were created using Python to gain deeper insights from the movie dataset. While visualization was not explicitly required for the project, we chose to include it as an additional layer of analysis to better understand complex trends and relationships within the data. Visualizing the data allows us to go beyond raw numbers and see patterns that are not immediately apparent through SQL queries alone. It also provides a more intuitive and accessible way to communicate the results of our analysis, which is critical when dealing with large datasets. The following visualizations showcase different aspects of the movie data and help us uncover key insights about the movie industry.

6.1 Rationale for Visualization

The decision to incorporate data visualization was driven by the desire to enhance the data analysis and to present the findings in a more comprehensive and engaging manner. The movie dataset, which includes a variety of attributes such as ratings, gross revenue, budgets, and directors, contains complex relationships that can be difficult to interpret using raw SQL queries alone. By visualizing these relationships, we are able to identify trends, outliers, and correlations that may not be immediately apparent. This helps in making the analysis more accessible and easier to understand, especially for stakeholders who may not be familiar with database management or SQL.

6.2 Distribution of Movie Scores

The first visualization we created focuses on the distribution of movie scores. To achieve this, we used a histogram along with a Kernel Density Estimate (KDE) curve. The histogram displays the frequency of movies falling within different score ranges, while the KDE curve smooths the distribution, providing a clearer understanding of how the scores are spread out. As seen in Figure 4, the highest frequency of movies falls within the 6 to 7 score range,

indicating that most movies tend to be rated moderately by audiences. This suggests that, generally, movies receive average reviews, with fewer outliers at both ends of the rating spectrum. The bell-shaped curve that emerges suggests a normal distribution, implying that movie ratings are fairly evenly spread across the available score range. This insight is valuable because it helps us understand the overall reception of movies, with the majority of films landing in the middle of the rating scale.

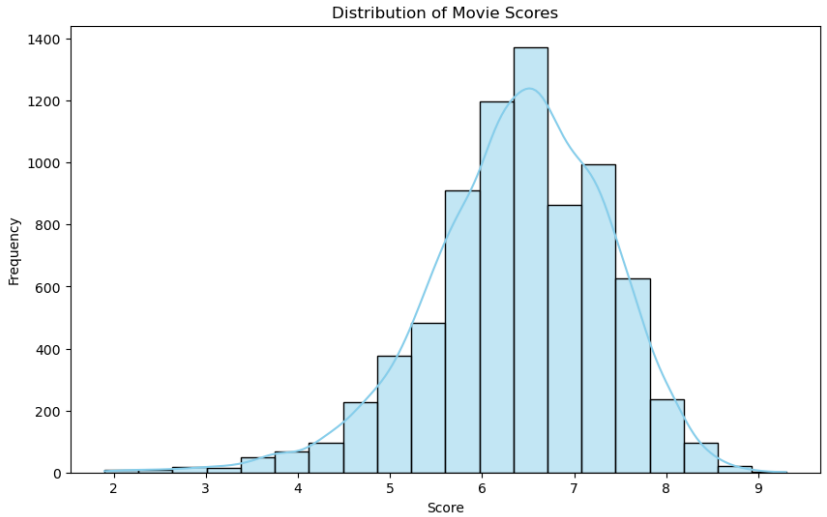


Figure 4: Distribution of Movie Scores with Kernel Density Estimate

6.3 Total Gross and Average Score by Year

The next visualization delves deeper into the relationship between total gross revenue and average movie scores over time. We used a dual-axis line plot to display two different trends on the same graph. The green line represents the total gross revenue, measured in billions of dollars, and it shows a steady upward trend starting from the 1980s. This trend aligns with the global expansion of the movie market and the increase in film production budgets over time. The blue line, representing the average movie score, fluctuates more significantly and stabilizes below 6.5 after the mid-2000s. This indicates that, despite the growing profitability of movies, audience satisfaction, as reflected in movie scores, has not increased at the same pace. The dual-axis plot is an effective way to compare two different metrics (gross revenue and average score) over the same period, and it reveals that while revenue continues to rise, the audience's average rating has remained relatively constant. This could suggest a gap between the increasing commercial success of movies and

the audience's evolving expectations.

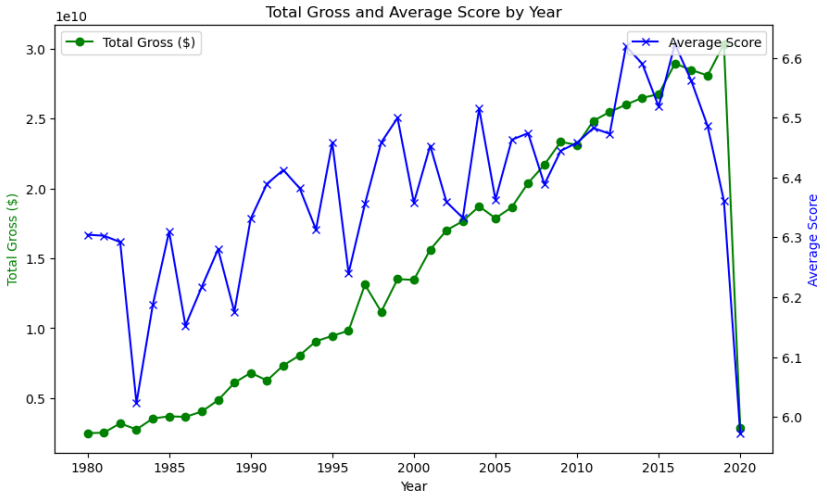


Figure 5: Total Gross Revenue and Average Score by Year

6.4 Gross Revenue vs. Budget

The third visualization explores the relationship between movie budgets and gross revenue. For this analysis, we created a scatter plot with points representing individual movies. These points are colored according to their average score, providing an additional layer of insight. As depicted in Figure 6, there is a clear positive correlation between budget and gross revenue: movies with larger budgets tend to earn more at the box office. However, this is not always the case, as some low-budget films achieve impressive gross revenues, suggesting that factors such as strong storytelling, star power, or audience demand can outweigh the impact of a film's budget. The color gradient is used to highlight that movies with higher scores are scattered throughout the entire budget range, indicating that high-quality movies are not solely the result of high production costs. Some lower-budget films outperform expectations both in terms of revenue and critical reception, showing that high ratings can come from films with more modest budgets.

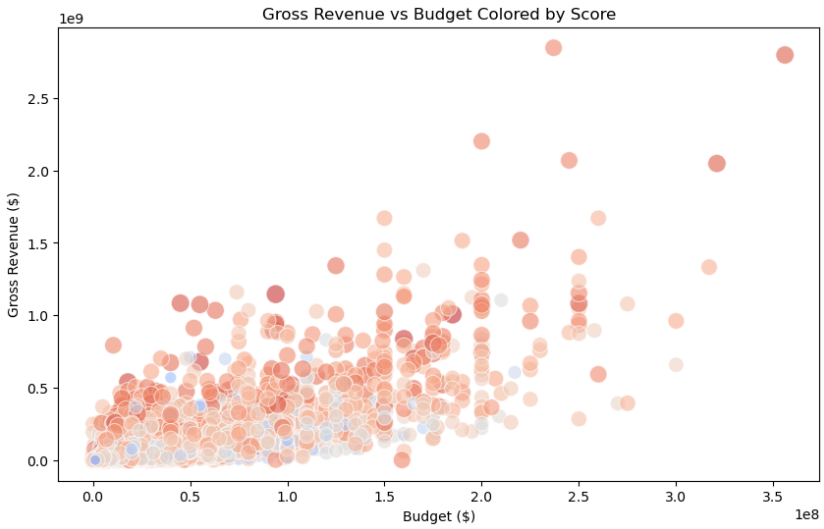


Figure 6: Gross Revenue vs. Budget Colored by Movie Score

7 Challenges Faced and Solutions

During the development of this project, we encountered several challenges, particularly when structuring and applying SQL queries to our movie management database. One of the primary issues arose when we realized that the `Movies` table was missing essential columns, specifically `Genre` and `Director`. These columns were critical for our analysis, as they provided key data on the movie's classification and creative leadership.

To address this challenge, we employed SQL queries to recover the missing data. Specifically, we used `JOIN` operations to link the `Movies` table with the `Directors` table to restore the `Director` information. Additionally, we connected the `Movies` table with the `Genres` table to retrieve the missing genre data. By doing so, we were able to re-establish the integrity of our database, ensuring that all relevant information was included and accessible for our analysis. This approach not only resolved the immediate issue but also demonstrated the importance of data integrity and the power of relational database management.

8 Conclusion and Future Improvements

In conclusion, this project has provided us with valuable hands-on experience in database design, data processing, and visualization. By utilizing SQL

for relational database management and Python for data analysis and visualization, we successfully designed and implemented a movie management system. We efficiently processed the data, conducted in-depth analysis, and visualized key trends, which helped to provide meaningful insights into the movie industry.

Through the application of relational storage and the enforcement of ACID compliance, we ensured that the database maintained its integrity, consistency, and reliability throughout the project. The project has significantly enhanced our skills in database querying, data processing, and visualization, and it has provided us with a deeper understanding of how to handle large datasets efficiently.

We are confident that the skills we gained from this project will be valuable in future endeavors, particularly in the fields of data science, database management, and data-driven decision-making. Thank you for your attention, and we look forward to applying these insights and techniques in future projects.